

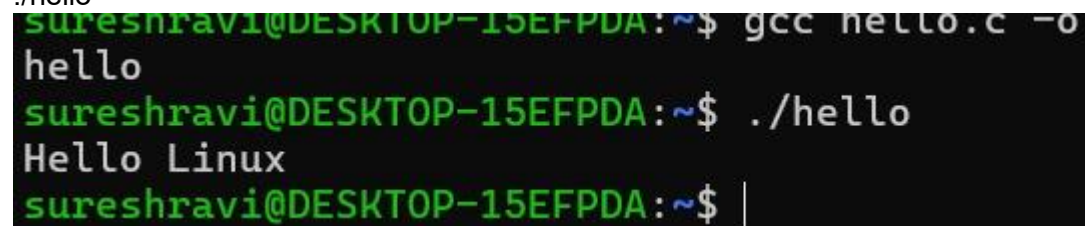
1. Hello.c

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello



```
sureshravi@DESKTOP-15EFPDA:~$ gcc hello.c -o
hello
sureshravi@DESKTOP-15EFPDA:~$ ./hello
Hello Linux
sureshravi@DESKTOP-15EFPDA:~$ |
```

2. Read User Name

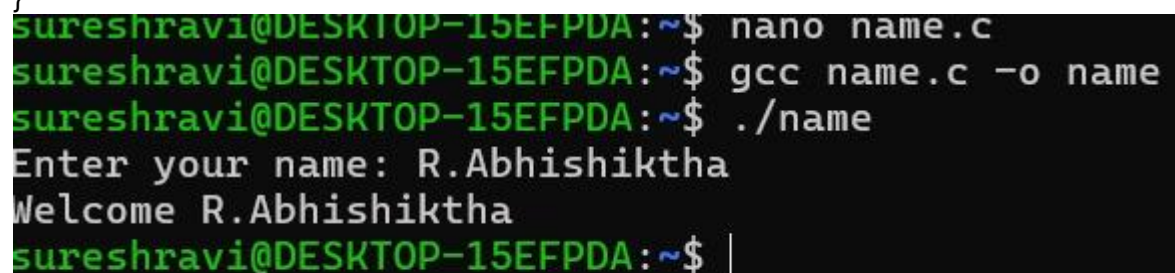
```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```



```
sureshravi@DESKTOP-15EFPDA:~$ nano name.c
sureshravi@DESKTOP-15EFPDA:~$ gcc name.c -o name
sureshravi@DESKTOP-15EFPDA:~$ ./name
Enter your name: R.Abhishiktha
Welcome R.Abhishiktha
sureshravi@DESKTOP-15EFPDA:~$ |
```

3. Display Process ID

```
#include<stdio.h>
```

```
#include<unistd.h>
```

```

int main()
{
    printf("Process ID = %d\n",getpid());

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ gcc pid.c -o pi
d
sureshravi@DESKTOP-15EFPDA:~$ ./pid
Process ID = 534
sureshravi@DESKTOP-15EFPDA:~$ |

```

4. Fork()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano fork.c
sureshravi@DESKTOP-15EFPDA:~$ gcc fork.c -o f
ork
sureshravi@DESKTOP-15EFPDA:~$ ./fork
Hello
Hello
sureshravi@DESKTOP-15EFPDA:~$ |

```

5. Parent and Child

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();
}

```

```

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano parent.c
sureshravi@DESKTOP-15EFPDA:~$ gcc parent.c -o
parent
sureshravi@DESKTOP-15EFPDA:~$ ./parent
I am Parent Process
I am Child Process
sureshravi@DESKTOP-15EFPDA:~$ |

```

6. Display parent and Child PID

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano pid2.c
sureshravi@DESKTOP-15EFPDA:~$
sureshravi@DESKTOP-15EFPDA:~$ gcc pid2.c -o p
id2
sureshravi@DESKTOP-15EFPDA:~$ ./pid2
Parent PID = 686
Child PID = 687
Parent PID = 686

```

7. Wait()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano wait.c
sureshravi@DESKTOP-15EFPDA:~$ gcc wait.c -o w
ait
sureshravi@DESKTOP-15EFPDA:~$ ./wait
Child Running
Parent Running After Child

```

8.Wait()

```

#include<stdio.h>
#include<unistd.h>

int main()
{

```

```

printf("Before exec()\n");

execlp("ls","ls",NULL);

printf("After exec()\n");

return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano exec.c
sureshravi@DESKTOP-15EFPDA:~$ gcc exec.c -o exec
sureshravi@DESKTOP-15EFPDA:~$ ./exec
Before exec()
ANU          exec.c.save  hello.c      parent      pid2        wait.c
example..txt fork          hello.c.save parent.c    pid2.c
exec         fork.c      name        pid         s2
exec.c       hello      name.c      pid.c       wait
sureshravi@DESKTOP-15EFPDA:~$ |

```

9.fork() + exec()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
}

```

```

sureshravi@DESKTOP-15EFPDA:~$ nano forkexec.c
sureshravi@DESKTOP-15EFPDA:~$ gcc forkexec.c -o forkexec
sureshravi@DESKTOP-15EFPDA:~$ ./forkexec
Tue Aug  4 14:05:37 UTC 2026
Child Completed
sureshravi@DESKTOP-15EFPDA:~$ |

```